

# SmartShelf

## Smart Retail Inventory & First-Expiry-First-Out (FEFO) Management System

---

**TEAM REPORT - PART 4 of 4** | Team Member 4: [Your Name]

**Your role on the project:** Engineering, security & deployment owner

**What YOU present and defend:** Authentication and security controls, the role-based access model, the data model, the test strategy, CI and the Docker deployment - plus the roadmap and growth opportunities.

### How to use this part

Read your sections below until you can walk someone through them without notes. Then practise the Q&A cues at the end - they mirror what a panel typically asks. If you do not know a detail outside your part, say so and hand it to the right teammate.

### You must be able to explain

- How passwords, sessions, 2FA, lockout and resets work.
- The RBAC model and the @at\_least rule.
- The database schema and the enums that drive the rules.
- The four test suites and what each proves.
- The CI pipeline and the Docker/TLS deployment.
- Design decisions, growth opportunities and the roadmap.

## 1. Authentication & security controls

---

- Passwords stored with bcrypt (adaptive, salted) - never plain text.
- TOTP two-factor authentication (GET /api/2fa/setup, enable/disable).
- Account lockout after 5 failed logins (15 minutes).
- IP rate limiting on login (10 attempts / 5 minutes).
- Strong session protection: sessions re-validated against IP and user-agent.
- Signed, single-use password-reset tokens (30-minute expiry).
- Full login history and an immutable audit log for every mutation.

## 2. Role-based access control

---

Roles are ordered USER < STAFF < CASHIER < BILLER < MANAGER < ADMIN. Endpoints declare @at\_least(Role.X) and grant anything at or above that rank. Only ADMIN can approve promotions or delete users; only MANAGER+ can touch master data, POs and stock adjustments.

## 3. Data model at a glance

---

| Domain      | Tables / values                                                                                                                         |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Identity    | users, login_logs (roles, bcrypt hash, 2FA secret)                                                                                      |
| Master data | categories, brands, products, suppliers                                                                                                 |
| Stock       | stock_batches, locations, location_transfers, inventory_movements                                                                       |
| Commerce    | purchase_orders(+items), customers, sales, sale_items, payments                                                                         |
| Pricing     | promotions (status lifecycle, discount type/value)                                                                                      |
| Ops         | notifications, audit_logs, wastage                                                                                                      |
| Enums       | Role, PurchaseStatus, SaleStatus, PaymentMethod, MovementType, DiscountType, WastageReason, NotificationType, ExpiryStatus, PromoStatus |

## 4. Test strategy (4 suites, all green)

---

| Suite               | Proves                                                                         |
|---------------------|--------------------------------------------------------------------------------|
| test_engines.py     | FEFO order, cross-batch allocation, risk thresholds, pricing advice            |
| test_security.py    | lockout, rate limiting, password policy, 2FA, reset tokens, session protection |
| test_smoke.py       | login, dashboard, receive, FEFO sale, alerts, reports, all pages               |
| test_concurrency.py | 20 parallel sales -> no oversell, exact stock end state                        |

Combined coverage is about 72%; the CI workflow enforces a 50% floor and runs the whole matrix on Python 3.11 and 3.12.

## 5. Deployment (Docker, one command)

---

- docker compose up starts MySQL 8.4 (healthchecked), the Flask app on gunicorn :8000 and an nginx TLS proxy (:80 -> :443).
- The entrypoint waits for the DB, runs init-db, optionally seeds demo data, then launches gunicorn.
- Self-signed certificates are generated for demos; production mounts a real cert volume.
- Scaling: move to managed MySQL/PostgreSQL and more gunicorn workers behind the proxy; the schema and API are already database-agnostic.

## 6. Design decisions, growth path, roadmap

---

**Decisions:** FEFO with an atomic guard is the load-bearing invariant; engines are rule-based and auditable; discounts require human approval; every invoice line is batch-accurate; new accounts are inactive until an admin verifies them.

**Growth path:** engines stay transparent and easy to audit - ML forecasting drops in later; notifications are lightweight polling today, push/websocket delivery later; dev runs on zero-setup SQLite and production scales to managed MySQL/PostgreSQL behind the TLS proxy.

**Roadmap:** receipt/invoice PDF printing, auto reorder drafts, server-side chart rendering for reports, barcode scanning at the POS, and a mobile manager view.

## 7. Likely questions and answer cues

---

### Q. How do you store passwords?

A. bcrypt with an adaptive cost and a per-user salt - never plain text or a reversible encoding.

### Q. How does 2FA work here?

A. The user requests /api/2fa/setup to get a TOTP secret, scans it into an authenticator app, then enables 2FA; logins then require the six-digit code.

### Q. How do you stop brute-force login attacks?

A. Account lockout after 5 failed attempts (15 minutes) plus IP rate limiting of 10 attempts per 5 minutes; both are covered by automated tests.

### Q. What is the concurrency guarantee and how is it tested?

A. Stock is deducted with one guarded atomic UPDATE (WHERE quantity >= n). The concurrency suite fires 20 parallel sales against one unit and proves stock never goes negative.

### Q. How would you deploy this for real?

A. docker compose up: MySQL + gunicorn + nginx TLS. For scale, managed databases and more workers; the ORM abstracts the DB choice.

### Q. Which endpoint protects the API from anonymous calls?

A. Flask-Login protects every route; API callers without a session get 401 JSON, page visitors are redirected to /login?next=...

### Q. How do you ensure data integrity?

A. One commit per transaction, guarded updates, foreign keys via the ORM, and an audit log written on every mutation.

### Q. What would you improve next?

A. Real chart exports, reorder automation, barcode support and multi-store / managed-DB scale - all already flagged on the roadmap.